



Angular a Web Framework

Web Framework: Crafted for Users, Powered by Google



"Angular is a web framework that empowers developers to build fast, reliable applications that users love."

– <https://angular.dev/>

We will build a small Angular application where:

- Parent contains employee data.
- Parent sends employee data to Child using @Input().
- Child displays the employee.
- Child has a Promote button.
- When clicked, Child sends an event to Parent using @Output().
- Parent receives the event and updates the employee designation and salary.

Scenario: Suppose we have an employee:

Employee Name : Rajesh Kumar
Department : IT
Designation : Software Developer
Salary : ₹60,000

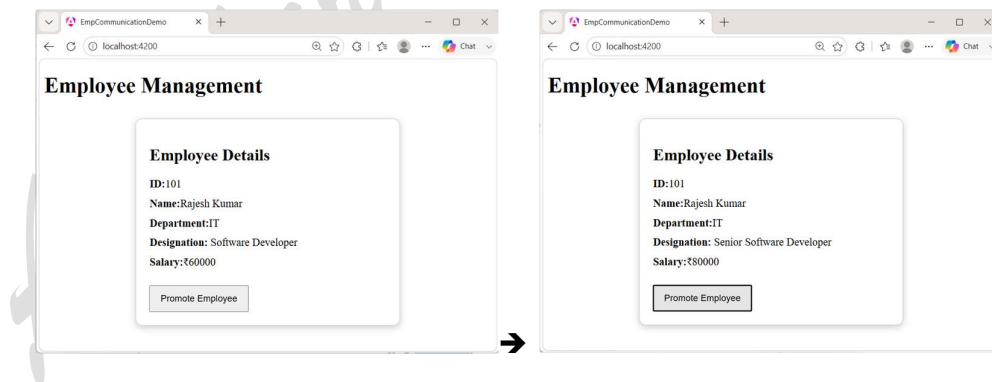
The Parent Component owns this data. The Child Component should only display the data.

AppComponent → Employee object → EmployeeComponent

When the child clicks a button: [Promote Employee] the child tells the parent:

"Employee has been promoted"

The parent then changes: Software Developer → Senior Software Developer



🔗 Connecting parts, sharing data, building apps. 📱



Presented by
Ranjan Bhatnagar



Angular a Web Framework

Web Framework: Crafted for Users, Powered by Google



Walk-through

1. Create a new Angular application: **emp-communication-demo**
2. Test run for default application: `ng s -o`
3. Create the Child Component: **employee**

`> ng g c employee`

CREATE src/app/employee/employee.spec.ts (568 bytes)
CREATE src/app/employee/employee.ts (205 bytes)
CREATE src/app/employee/employee.css (0 bytes)
CREATE src/app/employee/employee.html (24 bytes)

4. Our structure is:

AppComponent → @Input() → EmployeeComponent → @Output() → AppComponent

5. Create the Employee Model as a separate interface file name it: **EmpData.ts**

```
export interface EmpData {  
  id: number;  
  name: string;  
  department: string;  
  designation: string;  
  salary: number;  
}
```

6. Then define an employee object in App.ts (App Component – Parent)

```
employee: EmpData = {  
  id: 101,  
  name: 'Rajesh Kumar',  
  department: 'IT',  
  designation: 'Software Developer',  
  salary: 60000  
};
```

7. Make sure respective imports:

```
import { EmpData } from '../EmpData';
```

8. Send Data from Parent to Child Using @Input
9. Open employee.ts, here we need to receive the employee object.

```
@Input() employee!: EmpData;
```

10. Also add

```
@Output() promoteEmployee = new EventEmitter<EmpData>();
```





Angular a Web Framework

Web Framework: Crafted for Users, Powered by Google



```
promote() {  
  this.promoteEmployee.emit(this.employee);  
}
```

11. Resolve dependencies:

```
import { Component, EventEmitter, Input, Output } from  
'@angular/core';  
import { EmpData } from '../EmpData';
```

12. Now open: app.html, here we want to use the child component.

```
<h1>Employee Management</h1>  
<app-employee [employee]="employee"> </app-employee>  
<router-outlet />
```

13. Resolve Imports in .TS

```
imports: [RouterOutlet, Employee],
```

14. Display Employee in Child

```
<div class="employee-card">  
  <h2>Employee Details</h2>  
  <p>  
    <strong>ID:</strong>{{ employee.id }}  
  </p>  
  <p>  
    <strong>Name:</strong>{{ employee.name }}  
  </p>  
  <p>  
    <strong>Department:</strong>{{ employee.department }}  
  </p>  
  <p>  
    <strong>Designation:</strong> {{ employee.designation }}  
  </p>  
  <p>  
    <strong>Salary:</strong>₹{{ employee.salary }}  
  </p>  
  <button (click)="promote()">  
    Promote Employee  
  </button>  
</div>
```

15. Add Some Basic CSS to employee.css





Angular a Web Framework

Web Framework: Crafted for Users, Powered by Google



```
.employee-card {
  width: 350px;
  padding: 20px;
  margin: 30px auto;
  border: 1px solid #ccc;
  border-radius: 10px;
  box-shadow: 0 3px 10px #ccc;
}
.employee-card h2 {
  margin-bottom: 20px;
}
.employee-card p {
  margin: 10px 0;
}
button {
  margin-top: 15px;
  padding: 10px 15px;
  cursor: pointer;
}
```

16. Catch the @Output Event in Parent, modify app.html

```
<app-employee
  [employee]="employee"
  (promoteEmployee)="onEmployeePromoted($event)">
</app-employee>
```

17. Add the Parent Event Handler to app.ts

```
onEmployeePromoted(employee: EmpData) {
  console.log('Employee promoted:', employee);
  this.employee = {
    ...this.employee,
    designation: 'Senior Software Developer',
    salary: 80000
  };
};
```

18. Run and Test

 **Connecting parts, sharing data, building apps.** 



Presented by
Ranjan Bhatnagar